Complemento Práctico: Variables, Errores y Debugging

Secretos del entorno y manejo de memoria

En esta guía complementaria nos enfocamos en los detalles técnicos que surgen al escribir código en vivo. Analizaremos por qué NetBeans nos grita ciertos errores, la diferencia crítica entre declarar e inicializar, y cómo usar los comentarios como herramienta de ingeniería inversa y no solo como documentación.

Contenido

Debugging Manual

El uso estratégico de los comentarios.

Errores de NetBeans

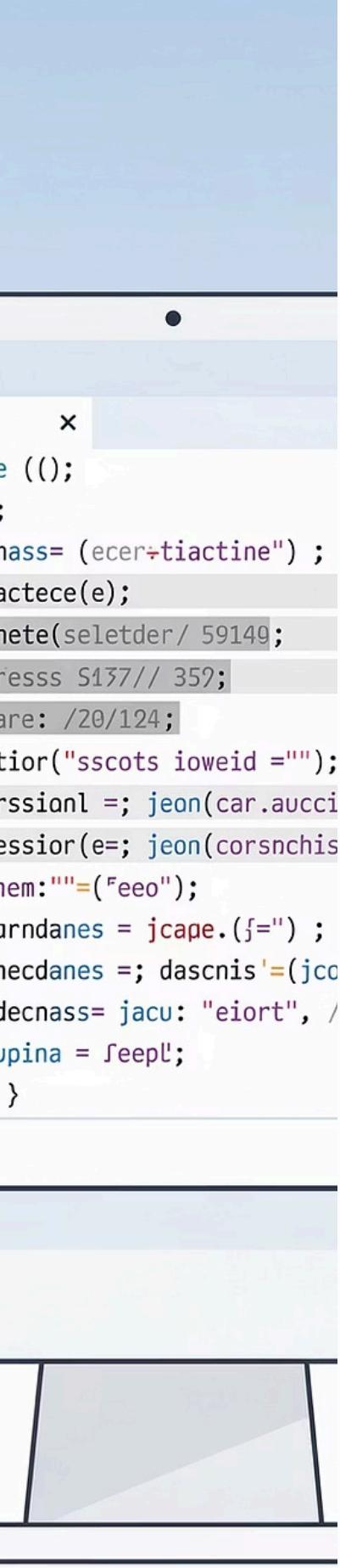
"Cannot find symbol" vs "Not initialized".

Tipos de Datos

Por qué usamos double y el misterio de la 'S' mayúscula en String.

Asignación

La trampa del signo igual.



Comentarios: Más que solo texto

El apunte PDF explica los tipos de comentarios (//, /* */), pero en la práctica profesional, los comentarios tienen un segundo uso vital: **Anular Instrucciones**.

Técnica de "Apagado" de Código

Cuando estamos probando un programa y algo falla, no siempre queremos borrar el código sospechoso (porque quizás estaba bien y luego queremos recuperarlo).

- **Acción:** Ponemos // delante de una línea de código ejecutable.
- **Efecto Visual:** NetBeans pintará la línea de **gris**.
- **Resultado:** El compilador ignora esa línea como si no existiera. Esto nos permite aislar errores sin destruir nuestro trabajo.

Interpretando a NetBeans: Errores de Variable

En el video vimos dos errores rojos que suelen confundir a los estudiantes. Es vital saber distinguirlos:

1

"Cannot find symbol" (No se encuentra el símbolo)

Causa: Estás intentando usar una variable que **no has declarado** (o escribiste mal su nombre).

Ejemplo:

```
// Error: la variable 'nombre' no existe todavía
System.out.println(nombre);
String nombre = "Pepe";
```

Solución: Verifica que la declaración (String nombre) esté **antes** del uso y que la ortografía sea idéntica (recuerda que Java distingue mayúsculas de minúsculas).

Declaración vs. Inicialización

1

"Variable might not have been initialized" (Podría no estar inicializada)

Causa: Declaraste la variable (la caja existe), pero está vacía. Java no permite usar cajas vacías.

Ejemplo:

```
String apellido; // Declaración (Reserva espacio)
// ¡Error! La caja está vacía
System.out.println(apellido);
```

La Analogía de los Lockers

Imagina que alquilas un locker (variable). Tienes la llave y el número. Pero si intentas sacar algo de adentro antes de haber guardado nada, es imposible.

- **Declarar:** Alquilar el locker.
- **Inicializar:** Guardar el primer objeto dentro.

La Elección del Tipo de Dato

El PDF lista byte, short, int, long. El video aclara una duda común: **¿Cuál uso?**

La Regla del int y el double

Aunque existen tipos más pequeños (byte) para ahorrar memoria, hoy en día la memoria es abundante.

Enteros

Usaremos **int** por defecto. Cubre hasta ±2 mil millones, suficiente para casi todo.

Decimales

Usaremos **double** por defecto, no float.

¿Por qué? Porque cuando escribes un número con coma en el código (ej: 10.5), Java asume automáticamente que es un double. Si intentas guardarlo en un float, te dará error de compilación a menos que agregues una 'f' al final (10.5f). Para evitar esa molestia, estandarizamos en double.

El Misterio de la 'S' Mayúscula

Un error recurrente en los exámenes es escribir string con minúscula.

- int, double, boolean van con **minúscula** (son primitivos, colores básicos de la computación).
- String va con **Mayúscula**.

¿Por qué?

Porque String no es un tipo primitivo; es una **Clase** (un objeto complejo). En Java, por convención, todas las Clases empiezan con mayúscula.

Tip de Experto

Notarán que String tiene métodos (funciones) propios, como .length() o .toUpperCase(). Un int no tiene "poderes", es solo un número. Un String es inteligente.

La Trampa del Signo Igual

En matemáticas, x = y significa que X e Y son equivalentes. En programación, **NO**.

Lectura Correcta (Derecha a Izquierda)

El operador = es destructivo.

```
edad = 30;
```

1

Cómo se lee

"Toma el valor 30 y guárdalo dentro de la variable edad, destruyendo lo que hubiera antes".

2

El flujo

La información viaja de derecha a izquierda.

3

Por qué importa

Esto explica por qué 30 = edad; daría error. No puedes meter la variable "edad" dentro del número "30".

Naming Conventions (lowerCamelCase)

Java es estricto pero también tiene reglas de etiqueta. Para nombrar variables usamos **lowerCamelCase** (camello minúsculo).

01

Empieza en minúscula

02

Cada nueva palabra empieza en Mayúscula

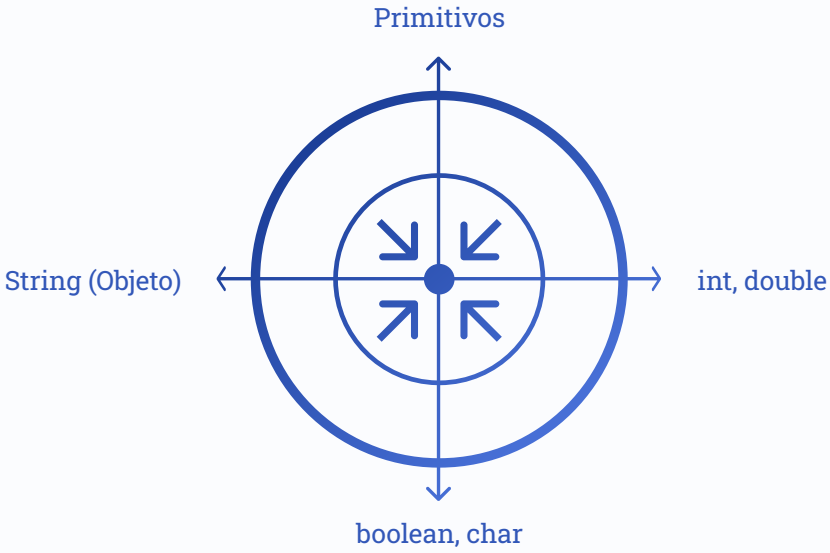
03

Sin espacios ni guiones

Ejemplos

`nombredeusuario`	❌	Mal: difícil de leer
`nombre_de_usuario`	❌	Mal: estilo de Python/PHP
`nombreDeUsuario`	✅	**Correcto: estilo Java**

Resumen de Tipos de Datos



Comprender la diferencia entre tipos primitivos y objetos es fundamental para dominar Java y evitar errores comunes de compilación.

¡Dominar estas sutilezas es lo que diferencia a un programador que "copia código" de uno que entiende la herramienta!

Debugging Manual

Usa comentarios estratégicamente para aislar errores sin destruir tu código.

Errores Comunes

Distingue entre "Cannot find symbol" y "Not initialized" para resolver problemas rápidamente.

Convenciones

Aplica lowerCamelCase y respeta las mayúsculas en String para escribir código profesional.